

File Sync

September 8, 2026

Overview

As we finalize the architecture for the P2P File Synchronizer, I've been thinking about the edge cases we will face when we actually implement the Merkle Tree and Vector Clock logic over a LAN. There are a few critical problems we need to resolve internally as a team, and a couple of API boundary clarifications we need to get from the Networking Team before we start writing code.

1. Clarifications Needed from the Networking Team

A. Peer Discovery Protocol

In a decentralized P2P setup, we don't have a central server holding everyone's IP address. When a new peer comes online, how do they find the others?

- **Action Item:** We need to ask the Networking team if they are handling UDP Multicast/Broadcast for peer discovery. If they are, what API method do we call to get the list of active peers? If not, we need to know if we are expected to use a hardcoded list of static IPs for testing.

B. Network Delivery Guarantees (TCP vs. UDP)

When our delta compression engine calculates a 4KB chunk of a file that needs to be synced, we will hand that byte array over to the Networking team's interface.

- **Action Item:** We need to clarify if they are using TCP (which guarantees delivery) or UDP (which does not). If they use TCP, we can assume our chunks arrive safely. If they use UDP, our File Sync team will have to design and implement a custom Retry/ACK (Acknowledgment) loop to ensure chunks aren't lost in transit.

2. Internal Logic to Resolve (File Sync Team)

C. The "Zombie File" Problem (Deletion Propagation)

Deleting a file in a distributed system is tricky. If Cop A deletes a file, and Cop B's laptop is closed, the system is fine. But when Cop B turns on his laptop tomorrow, his system will see that he has a file Cop A doesn't, and it might try to sync it back to Cop A, resurrecting the deleted file.

- **Proposed Solution:** We should discuss implementing "Tombstones". When a file is deleted, we don't delete its Vector Clock metadata. Instead, we flag it as a Tombstone so the system mathematically knows it was intentionally deleted, preventing it from coming back.

D. OS-Level File Locks

What happens if someone is actively typing in a document (e.g., MS Word), and a sync update arrives from another peer? The Operating System usually places an exclusive lock on the file while it is open, which will throw an `IOException` if our background sync tries to overwrite it.

- **Discussion Point:** We need to decide our fallback strategy. Do we create a temporary `.sync-conflict` file next to it? Or do we queue the chunk in memory and wait for the OS file lock to be released?

E. The “Late Joiner” Handshake (Cold Start)

Our chunking algorithm is great for small edits. But if two peers have been syncing 500 files for a week, and a brand-new peer joins the network with an empty directory, chunking doesn’t make sense.

- **Proposed Solution:** We need to design a “Full State Sync” handshake. When a node joins with an empty state (an empty Merkle Root), the system should bypass the chunking engine and initiate a full background transfer of all current files before normal delta-syncing resumes.